

Réalisation d'une action de tendance sur un asservissement de vitesse

Teddy Guillemot / Mewen Dutartre

L3 Génie Maritime, ENSM, Nantes

teddy.guillemot@supmaritime.fr / mewen.dutartre@supmaritime.fr

1-Formulation du problème

1-1 Problématique

L'objectif de cette étude d'automatique est de mettre en œuvre une action de tendance sur un asservissement de vitesse, en utilisant une carte OpenCM pour piloter un moteur via un hacheur afin de suivre précisément les consignes de mouvement. Pour concevoir une loi de commande performante, il est nécessaire de suivre une méthodologie qui commence par l'identification précise du système et sa modélisation numérique, pour obtenir un modèle nominal fiable. Cela permettra d'intégrer un correcteur dont on ajuste les gains avant de soumettre l'ensemble à des tests de robustesse en modifiant les paramètres, garantissant ainsi que la consigne reste atteignable malgré les variations du système réel. Nous allons dans ce projet utiliser Identity&Control, l'étude débute par une observation de la vitesse de rotation en boucle ouverte face à des commandes simples en pourcentage. Nous tenterons d'anticiper les résultats et interprétations attendues, puis de les confirmer ou de remettre en cause grâce à l'outil TelemetryViewer. Nous allons traiter deux consignes sinusoïdales et trapézoïdales. La démarche débute par une section théorique où sont détaillés l'identification du modèle nominal du système ainsi que les principes de fonctionnement de la loi de commande retenue grâce au logiciel. Enfin, l'étude progresse de la phase de simulation vers l'application réelle pour confirmer l'efficacité du suivi pour ces deux profils de vitesse distincts.

1-2 Hypothèses

Dans ce cadre, l'analyse fait abstraction des perturbations extérieures pour se concentrer exclusivement sur la qualité de la copie du signal.

La consigne de vitesse est supposée connue à l'avance et parfaitement définie. Cette hypothèse permet le calcul de ses dérivées temporelles nécessaires à la mise en œuvre de l'action de tendance par inversion dynamique. De plus, on suppose que la consigne est parfaitement suivie puisque la commande feedforward est directement la fonction inverse de celle du modèle.

Pour la consigne trapézoïdale, la consigne est considérée affine par morceaux, et les discontinuités de dérivée aux changements de pente sont négligées. La dérivée seconde est ainsi supposée nulle sur chaque intervalle, ce qui constitue une approximation acceptable au regard des contraintes physiques d'implémentation.

2- Méthode

2-1 Modèle nominal

En prenant en compte les trois dynamiques du système (une électrique et deux mécaniques), le modèle nominal a été établi sous la forme d'une fonction de transfert à trois pôles dans le domaine de Laplace. Ce modèle se caractérise par un gain $K_n = 0.644 \text{ rpm}/\%$ et trois constantes de temps $T_1 = 0.02\text{s}$, $T_2 = 0.029\text{s}$ et $T_3 = 0.029\text{s}$.

On a obtenu dans le domaine de Laplace : $H_n(p) = \frac{K_n}{(1+T_{n1}.p).(1+T_{n2}.p).(1+T_{n3}.p)}$

$$H(s) = \frac{0.64406}{(1+0.02s)(1+0.029938s)(1+0.029938s)}$$

$$H(s) = \frac{b_0 + b_1 s + b_2 s^2 + \dots}{a_0 + a_1 s + a_2 s^2 + a_3 s^3 + \dots}$$

Numérateur [b0 b1 ...]	0.64406	0	0	0
Dénominateur [a0 a1 a2 ...]	1	0.079877	0.0020938	1.7926e-05

2-2 Etude théorique du correcteur proportionnel-intégral

Le correcteur proportionnel-intégral (PI) est un régulateur couramment employé dans les systèmes d'asservissement, combinant deux actions complémentaires : une action proportionnelle et une action intégrale. L'objectif est d'assurer la réactivité et la précision.

L'action proportionnelle génère une commande (en %) directement proportionnelle à l'erreur, définie comme l'écart entre la consigne (en rpm) et la valeur mesurée. L'action intégrale, quant à elle, continue de faire évoluer sa commande (en %) tant que cet écart n'est pas ramené à zéro.

La première (u_p) confère à la boucle fermée sa rapidité de réponse, tandis que la seconde (u_i) en assure la précision et la robustesse.

En notant k_p le gain proportionnel (en %/rpm) et $\varepsilon(t)$ l'erreur (en rpm), la commande proportionnelle s'exprime de la façon suivante :

$$u_p(t) = k_p \cdot \varepsilon(t)$$

Avec k_i le gain intégral en (s^{-1}) on a la commande intégrale, et $T_i = \frac{k_p}{k_i}$:

Si le système traîne on pensera à diminuer T_i .

$$u_i(t) = k_i \cdot \int_0^t \varepsilon(j) dj$$

Or, dans un correcteur PI série, on a :

$$u_{pi}(t) = u_i(t) + u_p(t)$$

Ainsi, la commande du correcteur proportionnel intégral (en %) est :

$$u_{pi}(t) = k_p \cdot \varepsilon(t) + k_i \cdot \int_0^t \varepsilon(j) dj$$

et, $k_i = \frac{k_p}{T_i}$

Donc, $u_{pi}(t) = k_p \left\{ \varepsilon(t) + \int_0^t \frac{1}{T_i} \varepsilon(j) dj \right\}$

2-3 Comment paramétrer notre correcteur PI ?

On fixe le correcteur dans les schémas des simulations, pour trouver le paramétrage de PI, il faudra trouver les gains proportionnel et intégral optimaux pour le système. En établissant des conditions à respecter sur la mesure et la commande en fonction de la consigne, nous pouvons visualiser via Identity&Control notre paramétrage de notre correcteur PI.

Il ne faut pas que la commande sature et il faut que le temps de réponse à 5 % soit le plus faible possible. Nous avons ainsi réussi à déterminer les paramètres de notre correcteur PI.

2-4 Anti-reset Windup

Comme vu durant le dernier cours d'Automatique, il réside un problème avec la commande intégrale qui nous pousse à ajouter lors de la programmation ce qu'on appelle un « Anti-Reset Windup ». On ne le fera pas pour ce DM2 ci, en revanche il sera utilisé pour le DM3.

Tentons de résumer simplement ce qu'est le principe de l'Anti-Reset Windup. C'est l'idée que certes l'action intégrale permet d'assurer la robustesse du système mais elle peut aussi générer un phénomène d'emballement. Cela se produit si la consigne fixée est inatteignable pour le système dû à ses performances ou dû à des biais extérieurs. L'erreur sera alors fixe et non nulle et la commande intégrale va varier, sans jamais s'arrêter, que ce soit $\pm 100\%$. Ainsi, ça peut engendrer, lors d'un changement de consigne, une latence puisque la commande devra d'abord revenir dans sa plage de validité avant d'avoir l'effet escompté sur le système, souvent dans des temps long.

On peut donc penser parfois à une erreur de programmation en observant le système réagir, or ce peut être simplement dû à ce phénomène. L'Anti-Reset Windup sert alors à bloquer l'évolution de la commande intégrale au moment où elle atteint sa limite pour l'empêcher de la dépasser, et donc de limiter la latence

2-6 Etude de l'action de tendance

Pour la suite de cette étude, nous intégrons une commande appelée feedforward. Cette action de tendance a pour but d'anticiper la réaction du système afin que la vitesse du moteur suive la consigne de manière quasi instantanée. Techniquement, cela repose sur l'inversion du modèle nominal : on cherche à déterminer la commande nécessaire en amont pour obtenir l'effet désiré, bien qu'une telle causalité inversée soit purement théorique.

Cependant, comme cette méthode est principalement efficace pour le suivi de consigne et non pour contrer les perturbations imprévues, nous l'installerons en parallèle du correcteur PI, qui se chargera de la robustesse. L'intérêt réside aussi dans la validation de la loi de commande : en utilisant un modèle d'inversion légèrement

distinct du modèle nominal, nous testerons la stabilité du système face à des variations de paramètres. Si l'action de tendance s'avère concluante sur deux modèles différents, on peut raisonnablement supposer sa fiabilité pour une large gamme de scénarios réels.

Le modèle retenu pour cette commande prédictive s'appuie sur un gain $K_c = 0.616$ rpm/% et deux constantes de temps $T_1 = T_2 = 40ms$.

The screenshot shows a software interface with a purple background. At the top, the transfer function is displayed as $H(s) = \frac{0.61671}{(1+0.040153s)(1+0.040153s)}$. Below this, the general form of the transfer function is shown: $H(s) = \frac{b_0 + b_1s + b_2s^2 + \dots}{a_0 + a_1s + a_2s^2 + a_3s^3 + \dots}$. At the bottom, there are two input fields for the numerator and denominator coefficients. The numerator coefficients are $b_0 = 0.61671$, $b_1 = 0$, and $b_2 = 0$. The denominator coefficients are $a_0 = 1$, $a_1 = 0.080305$, and $a_2 = 0.0016122$.

Sa fonction de transfert est la suivante :

$$H_c(p) = \frac{N_c(p)}{U_{ff}(p)} = \frac{K_c}{(1 + T_{c1} \cdot p) \cdot (1 + T_{c2} \cdot p)}$$

Inversons à présent le modèle.

$$U_{ff}(p) = \frac{(1 + T_{c1} \cdot p) \cdot (1 + T_{c2} \cdot p)}{K_c} \cdot N_c(p)$$

Organisons l'équation de notre action de tendance.

$$U_{ff}(p) = \frac{1}{K_c} \cdot N_c(p) + \frac{(T_{c1} + T_{c2})}{K_c} \cdot p \cdot N_c(p) + \frac{(T_{c1} \cdot T_{c2})}{K_c} \cdot p^2 \cdot N_c(p)$$

Passons dans le domaine temporel. On obtient :

$$u_{ff}(t) = \frac{1}{K_c} \cdot N_c(t) + \frac{(T_{c1} + T_{c2})}{K_c} \cdot \dot{N}_c(t) + \frac{(T_{c1} \cdot T_{c2})}{K_c} \cdot \ddot{N}_c(t)$$

Dans notre étude, l'action de tendance sera réalisée à partir de cette équation en remplaçant la consigne $N_c(t)$ et ses dérivées (en rpm/s pour la dérivée simple et en rpm/s² pour la dérivée double) par nos fonctions trapèzes et sinus.

En revanche, on sait qu'il est impossible d'obtenir les dérivées en temps réel mais étant donné qu'on connaît les consignes et qu'elles sont de classe C^∞ donc dérivables une infinité de fois.

De plus, on constate que la commande feedforward n'est pas fonction de l'erreur mais de la consigne d'où son incapacité à assurer la robustesse de la boucle fermée.

2-7 Mise en place consigne sinusoïdale

Le premier objet d'étude est la mise en place d'une consigne sinusoïdale.

On va simuler le modèle nominal soumis à un correcteur PI dont les gains ont été trouvés précédemment.

Ensuite, nous écrirons les fonctions de la consigne et de ses dérivées afin de les implémenter à l'équation de l'action de tendance trouvée au préalable puis nous simulerons cette commande feedforward seule sur le modèle de commande utilisé puis sur le modèle nominal pour vérifier son fonctionnement et faire le test de robustesse. Enfin, nous combinerons les deux commandes, celle du correcteur proportionnel-intégral et celle du feedforward, pour faire une simulation puis pour l'implémenter dans la carte de commande à l'aide d'un programme écrit en Arduino.

2-7-1 Mise en place de l'action de tendance

L'action de tendance suit la consigne et anticipe la tendance que le système peut avoir face à une commande donnée. Pour vulgariser l'idée, on peut dire qu'elle diminue l'effet de la perturbation en agissant sur la commande avant que la perturbation n'ait d'effets.

Pour cela définissons l'action de tendance. Pour utiliser l'équation trouvée précédemment, on calcul la consigne et ses dérivées. On a une consigne sinusoïdale d'amplitude $A = 40 \text{ rpm}$ fréquence $f = 0,5 \text{ Hz}$:

$$\text{On a : } \omega = 2 \cdot \pi \cdot \left(\frac{1}{2}\right) = \pi \text{ rad} \cdot \text{s}^{-1} \text{ avec } N_c(t) = A \cdot \sin(2\pi f t)$$

$$\text{D'où, } N_c(t) = 40 \cdot \sin(\pi t), \dot{N}_c(t) = 40\pi \cdot \cos(\pi t), \ddot{N}_c(t) = -40\pi^2 \cdot \sin(\pi t)$$

Ainsi, nous avons :

$$U_{ff}(t) = \frac{40}{K_c} \cdot \{\sin(\pi t) + (T_1 + T_2) \cdot \pi \cdot \cos(\pi t) - \pi^2 \sin(\pi t) \cdot (T_1 \cdot T_2)\}$$

$$U_{ff}(t) = \frac{1}{K_c} \cdot 40 \cdot \sin(\pi t) + \frac{(T_1 + T_2)}{K_c} \cdot 40 \cdot \pi \cdot \cos(\pi t) - \frac{(T_1 \cdot T_2)}{K_c} \cdot 40 \cdot \pi^2 \sin(\pi t)$$

2-8 Mise en place consigne trapézoïdale

On a une consigne trapézoïdale est définie par :

- passage de -40 à $+40$ tr/min en 2 s
- palier à $+40$ tr/min pendant 2 s
- passage de $+40$ à -40 tr/min en 1 s
- palier à -40 tr/min pendant 3 s

La consigne $N_c(t)$ est donc une fonction affine par morceaux.

A présent on va définir la consigne et de ses dérivées.

Phase de montée ($0 \leq t < 2$ s) :

$$N_c(t) = -40 + 40.t ; \dot{N}_c(t) = 40 ; \ddot{N}_c(t) = 0$$

Palier haut ($2 \leq t < 4$ s) :

$$N_c(t) = 40 ; \dot{N}_c(t) = 0 ; \ddot{N}_c(t) = 0$$

Phase de descente ($4 \leq t < 5$ s) :

$$N_c(t) = 40 - 80t ; \dot{N}_c(t) = -80 ; \ddot{N}_c(t) = 0$$

Palier bas ($5 \leq t < 8$ s) :

$$N_c(t) = -40 ; \dot{N}_c(t) = 0 ; \ddot{N}_c(t) = 0$$

2-8-1 Mise en place de l'action de tendance

Or l'expression de l'action de tendance est :

$$u_{ff}(t) = \frac{1}{K_c} \cdot [N_c(t) + (T_1 + T_2) \cdot \dot{N}_c(t)]$$

$$\text{En phase de montée : } u_{ff}(t) = \frac{1}{K_c} \cdot [-40 + 40.t + (T_1 + T_2) \cdot 40]$$

$$\text{En palier haut : } u_{ff}(t) = \frac{40}{K_c}$$

En phase de descente : $u_{ff}(t) = \frac{1}{K_c} \cdot [40 - 80.t + (T_1 + T_2).(-80)]$

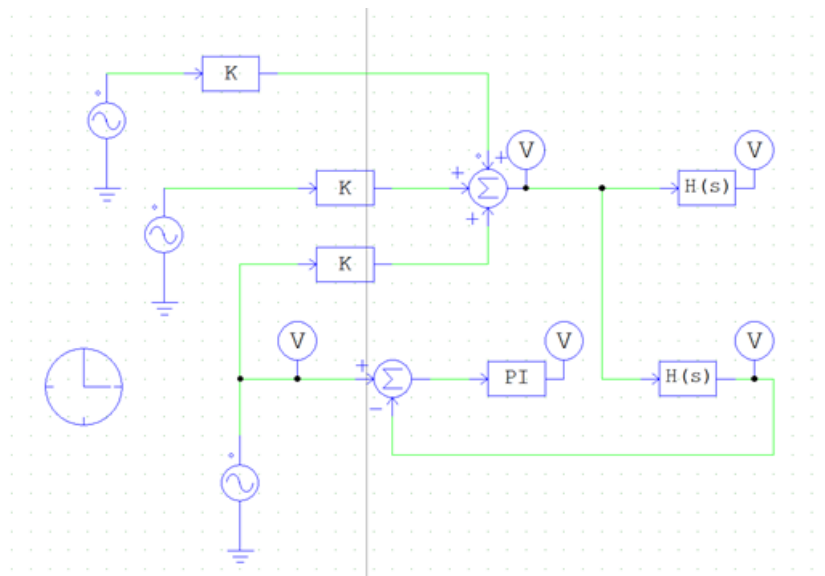
En palier bas : $u_{ff}(t) = \frac{-40}{K_c}$

Avec $K_c = 0.65 \text{ rpm}/\%$ et $T_1 = T_2 = 0.041 \text{ s}$

3- Résultats

3-1 Consigne sinusoidale

Nous allons utiliser le logiciel PSIM pour simuler nos lois de commande. Dans un premier temps nous avons utilisé le schéma de commande en boucle ouverte ci-dessous (la structure avec PI sera modifiée et utilisé pour la suite) :



Nous avons tout d'abord simulé l'action de tendance en utilisant uniquement le modèle H_c pour pouvoir la visualiser, et sans le correcteur PI. Nous pouvons déjà supposer que la sortie H_c suivra parfaitement la consigne car la commande feedforward est la fonction inverse du modèle H_c .

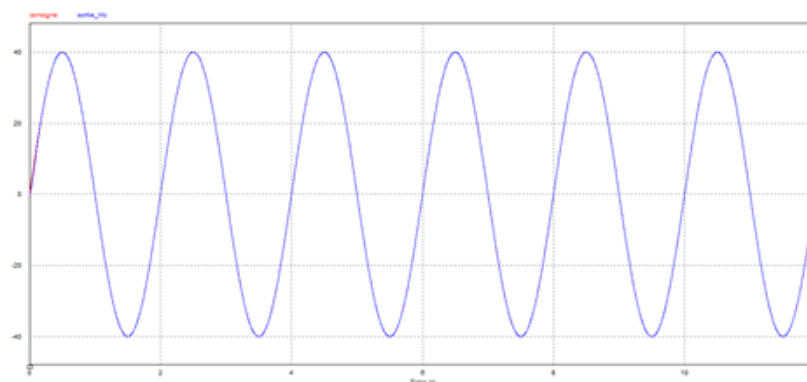


Figure 1. Graphique représentant la consigne et la sortie_Hc en fonction du temps

Nous pouvons ainsi bien voir que l'action de tendance permet de coller parfaitement à la consigne sinus comme nous l'avons annoncé. Cela permet de se rendre compte qu'elle fonctionne sur un modèle simple et qu'on va pouvoir ainsi l'appliquer au modèle H_n pour tester sa robustesse, toujours sans le correcteur PI. Nous pouvons prévoir que la mesure ne suivra pas aussi bien la consigne, mais restera très proche car nous avons des gains quasiment identiques : $K_c = 0,65 \text{ rpm}/\%$ et $K_n = 0,64 \text{ rpm}/\%$.

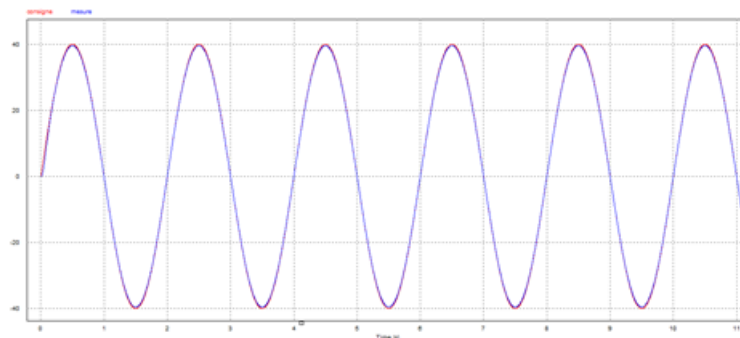


Figure 2. Graphique représentant la consigne et la mesure en fonction du temps

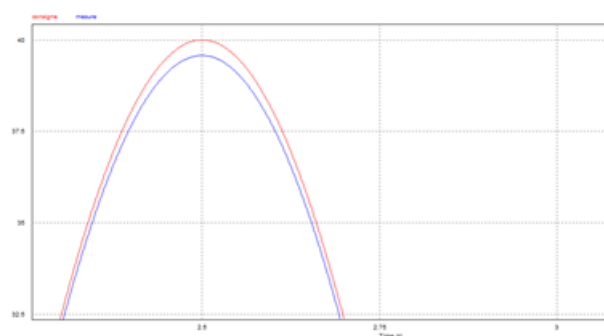


Figure 3. Zoom de la figure 2

Après simulation nous pouvons observer que la mesure est légèrement inférieure en valeur maximale et minimale, 2 rpm d'amplitude en moins par rapport à la consigne. Donc l'action de tendance fonctionne plutôt bien pour le modèle car la mesure suit très bien la consigne.

Nous allons donc maintenant simuler loi de commande complète, avec le PI et le feedforward sur le modèle nominal H_n . Nous espérons avoir une consigne et une mesure qui se suivront parfaitement. Pour cela nous allons utiliser le schéma de commande suivant qui est donc un montage en parallèle de l'action de tendance et du correcteur PI :

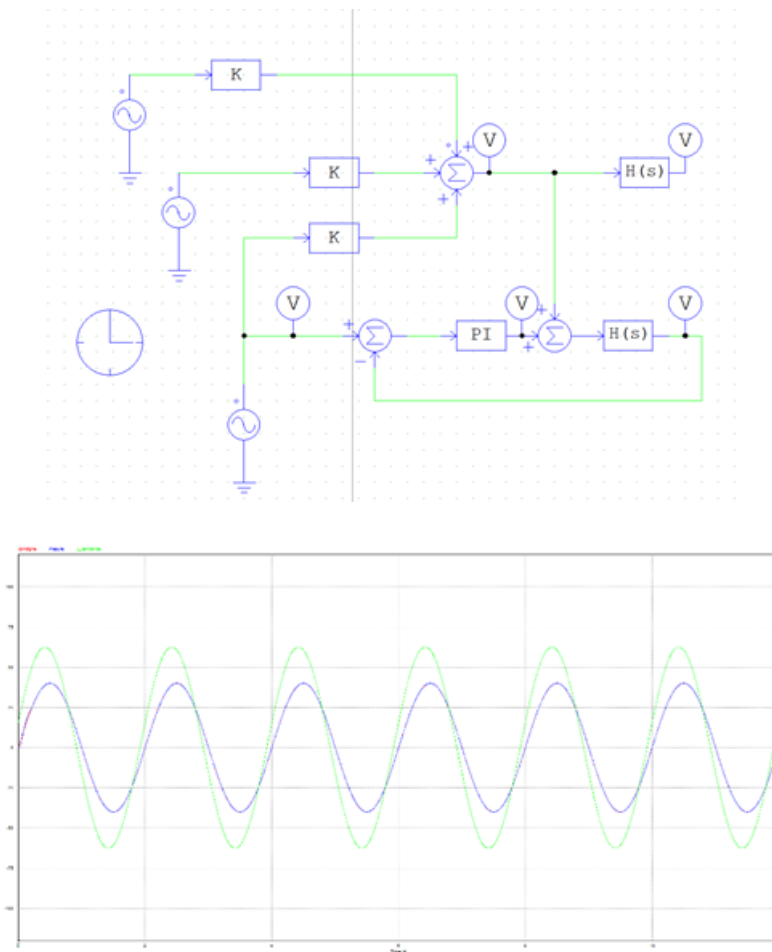


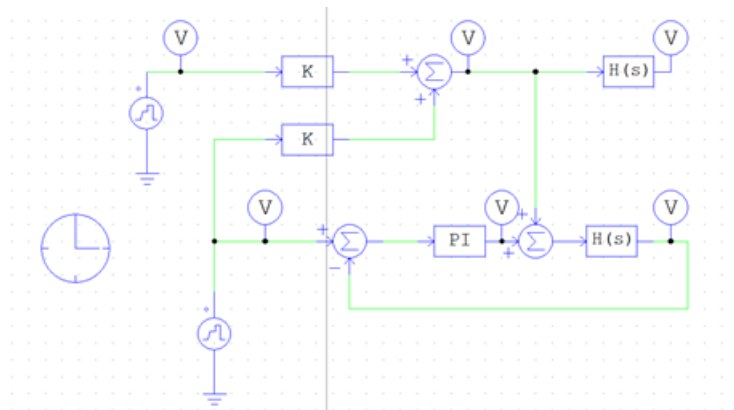
Figure 4. Graphique représentant la consigne, la mesure et la commande (en vert) en fonction du temps

Nous pouvons remarquer que la mesure suit bien de nouveau la commande. Grâce au correcteur proportionnel-intégral la différence d'amplitude est résolue. La commande, quant à elle est en avance sur la consigne ce qui correspond aux caractéristiques de l'action de tendance dont l'un des avantages est d'anticiper la consigne afin de pouvoir être le plus proche de cette dernière.

On peut donc conclure que la loi de commande complète fonctionne parfaitement et est robuste.

3-2 Consigne trapézoïdale

Nous avons ensuite réalisé ces mêmes étapes mais cette fois avec une entrée en trapèze que nous avons simulé avec le schéma de commande suivant :



Nous avons ici le signal créneau issu de la dérivée première uniquement car la deuxième n'est physiquement pas existante.

Pour la première simulation avec l'action de tendance seule sur H_c nous pourrions nous attendre, comme pour l'action sinus, à trouver une différence faible entre la mesure et la consigne.

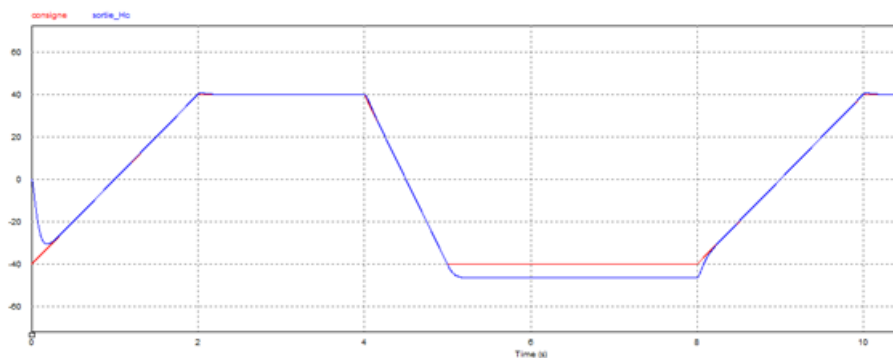


Figure 5. Graphique représentant la consigne et la sortie H_c en fonction du temps

Nous pouvons directement observer que sur la plus grande partie du tracer la sortie suit bien la consigne. Cependant, pour le passage à -40 rpm nous observons un net décalage entre les deux, cela est peut-être dû à notre paramétrage de notre deuxième source trapèze. Nous allons essayer de voir par la suite si cela a toujours un impact.

Nous allons donc maintenant faire la simulation avec le modèle nominal H_n et nous nous attendons à trouver une mesure qui suit bien la consigne sauf probablement sur la partie déjà non correcte de la première simulation.

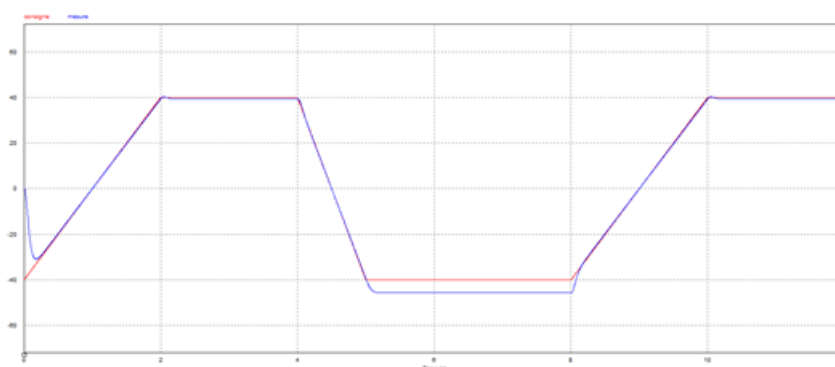


Figure 6. Graphique représentant la consigne et la mesure en fonction du temps

Nous trouvons donc bien, hormis la partie non cohérente, que la mesure suit parfaitement la consigne.

Nous allons donc maintenant passer à la dernière simulation qui est celle de l'ensemble correcteur PI et action de tendance, nous espérons avoir une consigne et une mesure qui se suivront parfaitement.

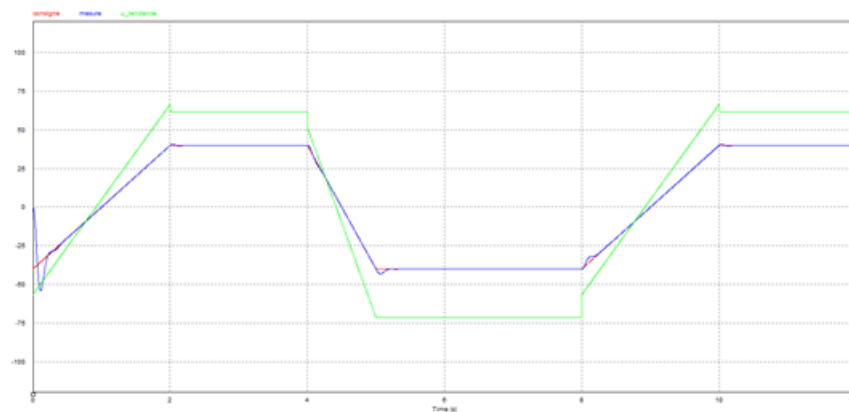


Figure 7. Graphique représentant la consigne, la mesure et la commande en fonction du temps

Nous pouvons tout de suite noter que nous n'avons plus d'écart entre la consigne et la mesure en -40 rpm alors que nous n'avons pas modifié les sources. Cependant nous avons en début de chaque pente une ondulation plus importante que précédemment mais qui peut encore être considéré comme négligeable.

On note aussi que pour la commande trapèze la commande n'est pas en avance sur la consigne et garde des variations plus importantes que la consigne pour pouvoir la suivre.

Nous remarquons aussi une saturation de la commande au début, et seulement au début de la simulation qui est lié à nos conditions initiales donc cela peut être négligé mais il est important de le prendre en compte pour d'autres système qui en serait sensible.

3-3 Implémentation

Nous allons maintenant implémenter les lois de commandes au système en utilisant le langage Arduino. On envoie donc dans un premier temps la loi de commande sinus pour faire tourner le moteur et ainsi mesurer sa vitesse de rotation et obtenir les graphiques suivants :

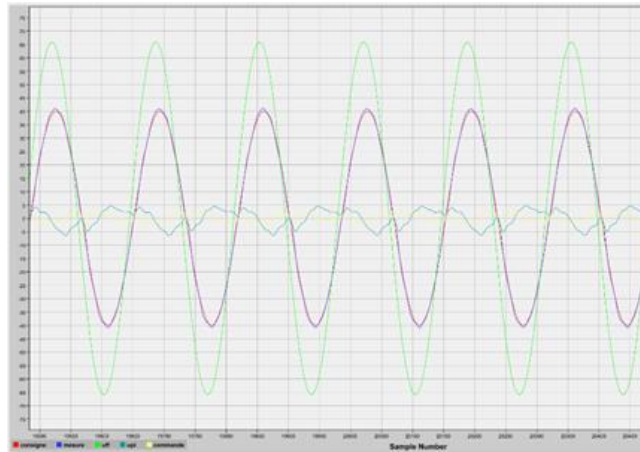


Figure 8. Graphique représentant la consigne, la mesure, la commande, le correcteur PI et l'action de tendance en fonction du temps

Nous remarquons que nous obtenons bien une simulation très similaire à celle obtenue avec PSIM. En effet la consigne est bien suivie même si nous pouvons noter un léger dépassement de la consigne qui est certainement dû à l'action intégrale du correcteur PI. De plus on note que l'action du feedforward est bien en avance pour "anticiper" et que la commande ne sature pas.

Nous allons maintenant implémenter la loi de commande trapézoïdale, nous nous attendons à ce que la consigne soit bien suivie par la commande et ayant sûrement quelques perturbations liées aux limites physiques du moteur.

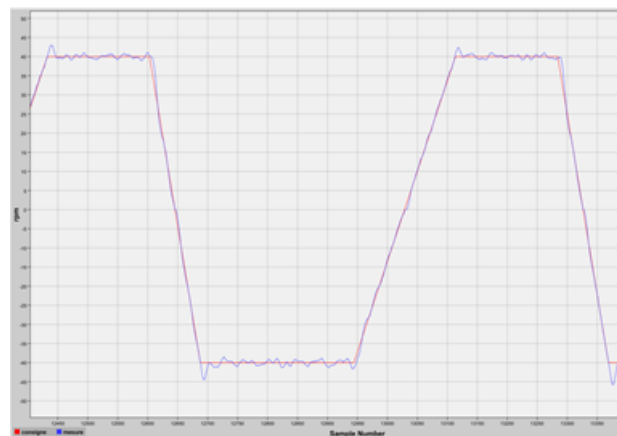


Figure 9. Graphique représentant la consigne et la mesure en fonction du temps

On remarque en effet qu'il y a un léger dépassement en fin de pentes puis que le moteur n'arrive pas à se stabiliser car la mesure oscille légèrement. Cette implémentation fonctionne plutôt bien mais on pourrait régler les gains pour réduire les perturbations.

4-Discussion

Il est important de rappeler nos hypothèses du début. Nous faisons abstraction des perturbations extérieures pour se concentrer exclusivement sur la qualité de la copie

du signal. De plus pour élaborer l'action de tendance, nous avons fait l'hypothèse que la consigne est parfaitement suivie puisque la commande feedforward est directement la fonction inverse de celle du modèle sans aucune variation de caractéristiques.

Les résultats obtenus mettent en évidence l'intérêt de l'action de tendance pour le suivi de consigne, aussi bien pour une consigne sinusoïdale que pour une consigne trapézoïdale. Dans les deux cas, l'ajout du feedforward permet d'anticiper la dynamique du système et d'améliorer significativement les performances de poursuite par rapport à un correcteur PI seul.

Dans le cas de la consigne sinusoïdale, l'action de tendance appliquée seule sur le modèle de commande H_c conduit à un suivi quasi parfait de la consigne, ce qui est attendu puisque la commande feedforward correspond à l'inverse du modèle utilisé. Lorsque cette même action est appliquée au modèle nominal H_n , un léger écart d'amplitude apparaît, directement lié à la différence entre les gains K_c et K_n , ce qui souligne la sensibilité de l'inversion dynamique à une erreur de modélisation. L'ajout du correcteur PI permet alors de compenser cet écart et d'assurer une poursuite précise, confirmant le rôle complémentaire du feedforward (anticipation) et du PI (robustesse).

Pour la consigne trapézoïdale, les performances sont globalement satisfaisantes mais mettent davantage en évidence les limites du modèle et de l'approche. En effet, la consigne trapézoïdale n'est pas de classe C^2 et présente des discontinuités de dérivée aux changements de pente. Dans cette étude, la dérivée seconde a été négligée, ce qui est justifié par son caractère non physiquement implémentable, mais cette approximation entraîne des transitoires plus marqués, visibles sous forme d'ondulations au début des phases de montée et de descente. Ces oscillations restent toutefois limitées grâce à la présence du correcteur PI.

Les phénomènes de saturation observés ponctuellement sur la commande, notamment en début de simulation, rappellent que l'inversion dynamique peut conduire à des commandes élevées lors de variations rapides de consigne. Dans un contexte réel, cela nécessiterait la mise en place de mécanismes complémentaires tels qu'un anti-windup ou une limitation explicite du feedforward afin de préserver l'intégrité du système et d'éviter des sollicitations excessives des actionneurs.

Enfin, cette étude confirme que l'action de tendance est particulièrement efficace pour le suivi de consignes connues à l'avance, mais qu'elle ne peut à elle seule garantir la robustesse face aux perturbations ou aux incertitudes de modèle. La combinaison feedforward + PI apparaît ainsi comme une solution équilibrée, permettant de concilier

anticipation dynamique et correction des erreurs résiduelles, ce qui en fait une architecture pertinente pour les systèmes d'asservissement de vitesse.

Références